

Technical Guide

Paycenter Web 4.0



2023_V1.3

Bancstac Confidential.

Table Of Contents

Table Of Contents.....	2
1. Introduction to Paycenter Web.....	3
2. Reference points for the intended audience.....	3
3. Paycenter Web features.....	3
4. Payment Configuration Options.....	5
4.1.Embedded Iframe.....	5
Why use an Iframe?.....	5
4.2 .Prepopulated Hosted Redirect Page (Prepopulated Hosted Payment Page).....	6
Why Use a Prepopulated Hosted Redirect?.....	6
4.3.Open Field Hosted Redirect Page (OpenField Hosted Payment Page).....	6
Why use an Open field Hosted Redirect?.....	6
4.4. Form Post.....	6
Why use a Form post?.....	6
5. Add ons and Additional Features.....	7
5.1. Tokenization.....	7
5.2. 3D Secure.....	8
Use with: Any payment method.....	8
Use with: Iframes or Hosted Pages.....	8
6. Transaction Processing.....	9
6.1. Iframe Page.....	9
6.2.Hosted Page.....	9
6.3.Form post.....	9
6.4.Payment Workflow.....	10
6.5. Payment Initialization message structure.....	11
6.6.Payment Init Request Fields.....	11
6.7. Payment init Response fields.....	13
6.8. Form Post Data Fields.....	14
6.9.Payment Complete Message Structure.....	14
6.9.1 Sample Payment_complete request generated by the API for you.....	14
6.10.PAYMENT_COMPLETE Response.....	15
6.10.1 Sample payment_Complete Response.....	16
7. Tokenization.....	17
7.1. Storing a card and obtaining a token.....	17
7.1.2. Performing a purchase and obtaining token simultaneously.....	18
7.1.3. Tokenization Responses.....	19
8. Making Payments Using Tokens.....	20
Support and Troubleshooting.....	21

1. Introduction to Paycenter Web

This guide provides technical information about integrating and configuring Paycorp's Paycenter Web Solution to your website or application. Paycentre Web v.4.0 allows merchants to process credit card payments in a PCI DSS compliant manner.

2. Reference points for the intended audience

This guide is intended to act as a reference point for web/application developers seeking to implement Paycorp transaction functionality into merchant websites and for web applications using the Paycorp API for Paycorp's REST web services.

Our API will generate the json messages for you and handle marshaling and unmarshaling of json messages so that the merchant does not have to spend time writing code for these tasks.

Our Sample codes are currently available in Java, .NET and PHP. However if you are using any other language you may directly integrate into our REST web services.

It is assumed that the parties implementing this product will be experienced in the development and implementation of websites and web applications, this document is written principally toward parties with such expertise. Therefore In Particular you should understand,

- JSON based Web Services
- Https protocols

3. Paycenter Web features

- Fully customized, responsive payment pages.
- Card Tokenization can be enabled here for token creation
- Sample code provided for easy integration resources for your reference

When setting up an instance of a Payment Page, you will be provided with the following information & resources:

- This Documentation
- Client Id
- Authtoken
- HMAC Secret
- End point
- Response codes
- Sample Cards

We will require the following information from you:

- The Location of any CSS you want to be applied to an iframe or Hosted Page By Default, an advanced Payment Page instance is set up with the following Configuration.

Parameter	Value
Permitted Card Types	Visa,Master Card, Amex
Page Type	Iframe
Transaction Type	Purchase
CVV Required	True
Tokenize	True
3D Secure	Disabled
Surcharging	Disabled

PLEASE ENSURE YOU USE THE CORRECT CLIENT ID, AUTHTOKEN, HMAC SECRET AND ENDPOINT AS PROVIDED TO YOU BY PAYCORP.

4. Payment Configuration Options

The Paycentre Web v.4.0 supports a wide variety of configurations to suit most Merchant Implementations. Review the below options to consider which implementation style suits your requirements.

Before you do the integration, please check that the server is having TLS 1.2 protocol enabled and having the compatible curl version 7.3.0 or above.

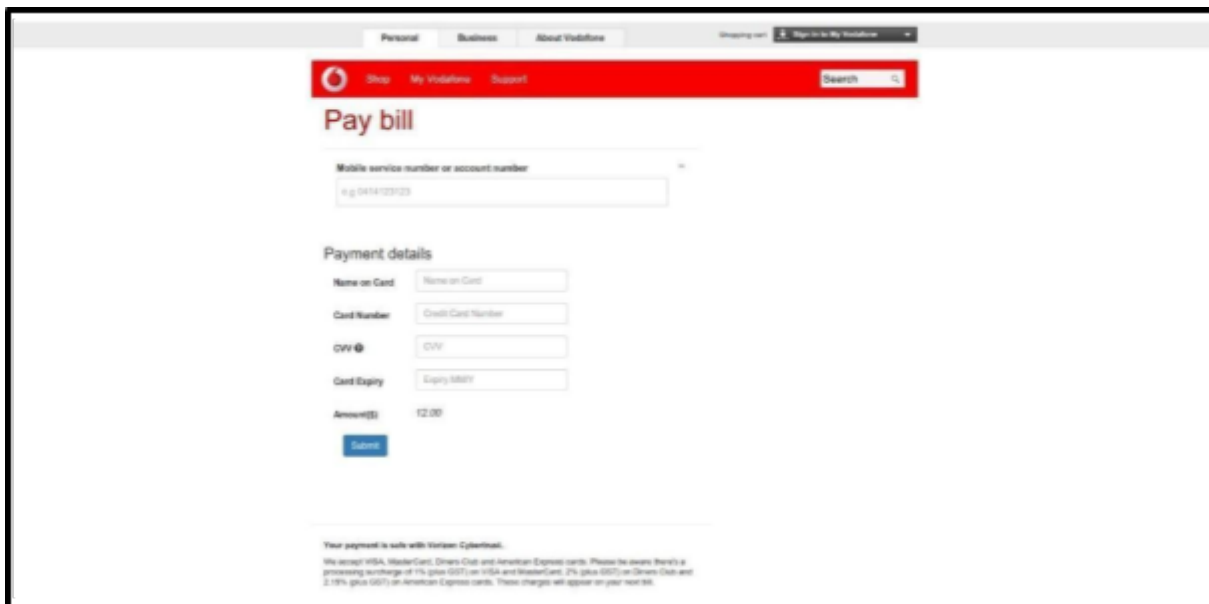
4.1.Embedded Iframe

Why use an Iframe?

Iframes are a good blend of security and compliance while causing minimal disruption to page layout. Iframe based payments are compliant according to PCI DSS v3, and can significantly reduce scope for PCI compliance, unlike a Form Post or API. Iframe is the only method to keep your web service out of scope while maintaining a seamless appearance to your payment page.

Paycorp's iframe option is highly customizable, you can bring your own CSS and even HTML to fully customize the page.

Example page with embedded iframe



The screenshot shows a web page titled "Pay bill" with a red header bar containing navigation links: "Personal", "Business", "About Vodafone", "Shopping cart", and "Sign in to My Vodafone". Below the header, there is a search bar and a "Pay bill" title. The main content area contains a form for payment details. The form includes a field for "Mobile service number or account number" with a placeholder "e.g. 0474 123 123". Below this, there is a section titled "Payment details" with fields for "Name on Card", "Card Number", "CVV", and "Card Expiry". The "Amount" is displayed as "12.00". A blue "Submit" button is at the bottom of the form. At the bottom of the page, there is a small disclaimer: "Your payment is safe with Vodafone CyberShield. We accept VISA, MasterCard, Diners Club and American Express cards. Please be aware there's a processing surcharge of 1% plus GST on VISA and MasterCard, 2% plus GST on Diners Club and 2.19% plus GST on American Express cards. These charges will appear on your next bill."

4.2 .Prepopulated Hosted Redirect Page (Prepopulated Hosted Payment Page)

Why Use a Prepopulated Hosted Redirect?

The Prepopulated Hosted Redirect brings the user off the merchant website and onto a discrete secure payment page hosted by Paycorp. The Prepopulated Hosted Redirect has many of the same advantages of the iframe, being highly secure, fully PCI compliant and customizable using CSS and HTML. This can be a good option for small merchants wishing to leverage the trust and brand visibility of Paycorp.

4.3.Open Field Hosted Redirect Page (OpenField Hosted Payment Page)

Why use an Open field Hosted Redirect?

The Difference between the Open Field Hosted Redirect page and the Prepopulated one is that the metadata fields remain open to be entered by the cardholder. This can be a simpler implementation for small merchants. This is also useful where you need the cardholder to enter information about their payment themselves.

4.4. Form Post

Why use a Form post?

This option is only for PCI compliant merchants only. The Form post gives the Merchant the most control over the Customer Experience when making their payment. However, this comes at the significant cost of bringing the entire web server, and its associated devices and servers on the same network into PCI DSS scope. Form post is recommended only where the Merchant is already PCI DSS certified.

5. Add ons and Additional Features

In addition to the fundamental payment process, the Advanced Payment page also allows for a number of separate features.

5.1. Tokenization

In addition to processing Payments directly, Paycentre Web v.4.0 includes a tokenization service which enables you to store credit cards in Paycorp's secure encrypted vault . Here the card is stored in Paycorp's encrypted PCI compliant servers and we issue the merchant a code called a Token which represents the stored card.

Paycentre Web v.4.0 can be used for storing cards with Paycorp's secure CardVault and obtaining tokens. However in order to process tokens you must use Real Time payments .

The Paycorp tokens are numeric non-mod10 values which retain the same first 6 and last 4 digits of the credit card they replace. There is a 1 to 1 relationship between the token and the card.

This enables the merchant to use the token in the same fields and locations as a credit card, displaying a masked token will look the same as a card and they will fit in the same database fields as a real card.

We will discuss Tokenization and making payments using Tokens in detail in section 08.

5.2. 3D Secure

Use with: Any payment method

3D Secure is an additional security layer which allows the Issuer or the card to check that the transaction about to take place isn't fraudulent before the actual payment takes place. The other major advantage of 3D secure is that the chargeback liability can be shifted away from the Merchant and back to the issuer. This is a great tool for high-risk payments or where security is paramount. The disadvantage of 3D secure is that the process can disrupt the checkout process.

Speak to your Paycorp Implementation Agent to get more information about how 3D secure can help your business.

Use with: Iframes or Hosted Pages

One of the key advantages of Paycorp PCI Compliant payment solutions is that they remain PCI compliant whilst being highly customizable and integrate seamlessly into your own website.

CSS: The Paycorp page can reference an externally available CSS file hosted in the merchant environment. This way there is no need to contact Paycorp for any minor CSS updates.

HTML: Paycorp can also host a HTML template file to further customize the iframe or Hosted page, ensuring the page fits within your style guides and branding. For security purposes, this needs to be hosted by Paycorp, though can be easily updated.

6. Transaction Processing

Paycorp advanced payment pages offer three technical methods of processing transactions for merchants.

6.1. Iframe Page

Merchants will embed a paycorp hosted page as an iframe to their website. Merchants can host css for iframe, so they have full control over the look and feel of the page. A Merchant may also supply Paycorp with a HTML template to improve page seamlessness.

6.2.Hosted Page

Merchants will redirect the payer to paycorp hosted page. Paycorp url will be visible on the page. Like the iframe page, this method also enables merchants to have full control over the look and feel of the page.

6.3.Form post

Merchants who are already PCI DSS compliant can manage their own html and post payer's card information to Paycorp.

Note: We recommends the Iframe or Hosted solution for most Merchants

6.4.Payment Workflow

The page is a highly customizable PCI compliant Payment that uses 02 JSON based web service calls and one https call in order to complete the Payment Request.

The **First** call is the **Payment Initialization**.

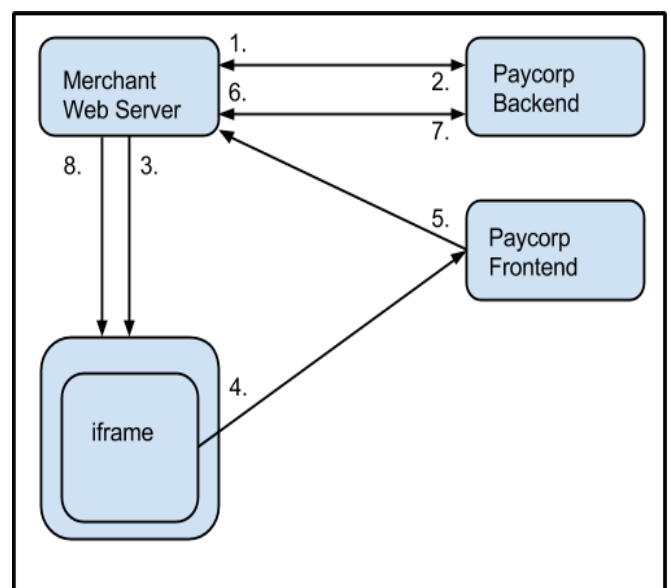
The response to this message is the **Paymentinit Response**. This response contains the **paymentPageUrl**. This is the URL you would embed in the iframe or redirect to in the case of a redirect method (Hosted payment pages). For the iframe option you will embed the URL as follows,

```
<iframe src="value of the paymentPageUrl goes here "></iframe>
```

After the card data is entered by the customer in the capture page and submitted, the merchant will receive a response to the Receipt URL specified in the Payment Initialization request '**Redirect**' parameter. This response will be via http GET with a parameter called ReqID.

The **Second** Call is the **Payment Complete call**. This call is the trigger to submit the full bundle comprising the first two calls to the Merchant's acquiring Bank. The Merchant submits the ReqID and is returned with a full transaction response, including any/all metadata sent in the first Call, the Token, if it was requested, sanitized version of the cardholder data (truncated card, expiry date, no CVC). At this point the payment is complete.

1. Merchant submit Payment_Init call
2. Paycorp responds back with ReqID & Payment URL
3. Merchant's web server delivers parent page to Browser, including the Payment URL embedded as an iframe
4. Cardholder completes payment, cardholder data captured by Paycorp.
5. Response returned to Receipt URL specified in Call 1.
6. Merchant submits Payment_complete call
7. Paycorp confirms payment processed, returned full transaction bundle, including token if any
8. Merchant Web server serves browser with receipt pagePayment Initialization message structure



6.5. Payment Initialization message structure

This method initializes a payment and returns a unique request id. Operation: PAYMENT_INIT

6.6.Payment Init Request Fields

Property	Type	Description
clientId	Integer	ClientId provided by paycorp required .
type	String	Options: PURCHASE, AUTHORISATION, TOKEN Default: PURCHASE
tokenize	Boolean	Flag to tokenize card with PURCHASE and AUTHORISATION. If you only want to tokenize use type as TOKEN.
tokenReference	String	Reference used in conjunction with a tokenization request..
amount	Object	Transaction amount and currency. paymentAmount: Type decimal currency: 3 char ISO code Required. Amount / Currency code - ISO LKR USD
redirect	Object	Set of redirect URLs you provide for paycorp to redirect payers to the merchant website. returnUrl: The payer is redirected to this URL after submitting the payment. ReturnMethod:
clientRef	String (50 Chars Max)	Merchant specified field
comment	String (100 Chars Max)	Merchant specified field
extraData	Map(100 Chars Max)	Merchant specified key value pair metadata.

Sample Payment_INIT Request generated by the API for you

```
POST /paycorp-webservice/InterfaceServlet HTTP/1.1
Host: https://sampath.paycorp.lk/rest/service/proxy
AUTHTOKEN: d5c1b696-b86a-4613-92f8-62dd412b1a56
Content-Type: application/json
Cache-Control: no-cache
{
  "version": "1.5",
  "msgId": "AD32B8FC-0D72-41D3-8F6B-51FB2107835E",
  "operation": "PAYMENT_INIT",
  "requestDate": "2023-07-10T07:50:43.920+0530",
  "validateOnly": false,
  "requestData": {
    "clientId": "14000396",
    "clientIdHash": "",
    "transactionType": "PURCHASE",
    "transactionAmount": {
      "totalAmount": 0,
      "paymentAmount": 200,
      "serviceFeeAmount": 0,
      "currency": "LKR"
    },
    "redirect": {
      "returnUrl":
      "cancelUrl": "",
      "returnMethod": "GET"
    },
    "clientRef": "",
    "comment": "",
    "tokenize": false,
    "cssLocation1": "",
    "cssLocation2": "",
    "useReliability": true,
    "extraData": "{1,2,3}"
  }
}
```

Note: To get the successfull response in the test environment you need to put at least 200(2.00) (your currency) as PaymentAmount. And also decimal values not supported as payment amount in testing environment.

Ex:

Use Amount Values like: 200 , 300 , 5000 etc.

6.7. Payment init Response fields

Property	Type	Description
reqid	Integer	A unique payment id provided by paycorp.
expireAt	String	Time at which this request will expire. Currently valid for 30 mins.
paymentPageUrl	Object	Url to get Iframe and hosted payment pages, not used for form post solutions.

Sample Payment_INIT Response raw json message

```
{
  "msgId": "AD32B8FC-0D72-41D3-8F6B-51FB2107835E",
  "responseData": {
    "reqid": "Z02W1pFzT5mxhR3SAyCw",
    "expireAt": "2023-07-10T11:58:59.591+0000",
    "paymentPageUrl":
      "https://paycorp-smp.prod.aws.paycorp.lk/webinterface/app/payment?reqid=Z02W1pFzT5mxhR3SAyCw"
  }
}
```

Note: It is strongly recommended to implement additional validation during the integration process, specifically incorporating the use of client references and request IDs. While the client reference is an optional parameter provided to the merchant for assistance, the request ID is a unique identifier generated for each payment transaction. We emphasize the importance of validating both client references and request IDs from the merchant end to enhance the security and integrity of the integration.

6.8. Form Post Data Fields

The form post method is only for Merchants who are PCI compliant. Fields for form post request.

After submitting, paycorp will store payment details and redirect payer to return url using return method as provided in payment init request. Only parameters which will pass in the return url will be reqid.

Example: `http://some_merchant.com/confirm?reqid= 5bDqiptxShatUDb5kzIY`

6.9.Payment Complete Message Structure

Completes payment and returns final transaction response. Operation:

PAYMENT_COMPLETE

Payment Complete Request Fields

Property	Type	Description
clientId	Integer	ClientId provided by paycorp Required .
reqid	String	Request Id provided by paycorp in payment_init request.

6.9.1 Sample Payment_complete request generated by the API for you

```
POST /paycorp-webservice/InterfaceServletHTTP/1.1
Host : https://sampath.paycorp.lk/rest/service/proxy
AUTHOKEN: d5c1b696-b86a-4613-92f8-62dd412b1a56
Content-Type: application/json
Cache-Control: no-cache
{
  "version": "1.5",
  "operation": "PAYMENT_COMPLETE",
  "msgId": "AD32B8FC-0D72-41D3-8F6B-51FB2107835E",
  "requestDate": "2023-07-10T16:32:43.920+0530",
  "validateOnly": false,
  "requestData": {
    "clientId": "14000396",
    "reqid": "ruBJhNTWRiOmygLdbN2k"
  }
}
```

6.10.PAYMENT_COMPLETE Response

Property	Type	Description
clientId	Integer	ClientId provided by paycorp.
type	String	Transaction type as provided in payment init request
tokenize	Boolean	Flag to tokenize card with PURCHASE and AUTHORISATION transaction type
tokenReference	String	
creditCard	object	Credit Card info submitted by payer type holderName number expiry

amount	object	Transaction amount in CENTS and currency type as submitted by merchant in payment init request. E.g: initRequest.transactionAmount = new TransactionAmount(1010,"AUD"); Please see currency codes given below.
clientRef	String	Merchant specified field
comment	String	Merchant specified field
extraData	String	Merchant specified key value pair metadata.
txnReference	Integer	Response Field - Paycorp transaction reference
responseCode	String	Response Field - Bank Response Code
responseText	String	Response Field - Bank Response Test

6.10.1 Sample payment_Complete Response

```
{
  "msgId": "AD32B8FC-0D72-41D3-8F6B-51FB2107835E",
  "responseData": {
    "clientId": 14000396,
    "clientIdHash": "2C5rossdaTq84aU1DGZJFoK92M",
    "transactionType": "PURCHASE",
    "creditCard": {
      "type": "VISA",
      "holderName": "test",
      "number": "456445***4564",
      "expiry": "0926"
    },
    "transactionAmount": {
      "totalAmount": 0,
      "paymentAmount": 200,
      "serviceFeeAmount": 0,
      "withholdingAmount": 0,
      "currency": "LKR"
    },
    "clientRef": "Test",
    "comment": "test",
    "extraData": {},
    "txnReference": "2023200009621577",
    "responseCode": "00",
    "responseText": "TRANSACTION APPROVED",
    "tokenized": false,
    "settlementDate": "2023-08-22",
    "authCode": "826233",
    "cvcResponse": "",
    "authenticationValue": "AAUDA0WHZgAAAADIFEI0dQAAAAA=",
    "dsTransID": "cfbf2764-126a-4b0e-ada5-e3d580ff7291",
    "threeDSServerTransID": "59f81576-1ea2-48f6-ab86-b242345391a6",
    "transStatus": "Y",
    "transStatusCode": "",
    "transStatusDescription": "",
    "eci": "05"
  }
}
```


7. Tokenization

7.1. Storing a card and obtaining a token

7.1.1. Tokenization Request

○ JAVA

```
PaymentInitRequest initRequest = new PaymentInitRequest();
initRequest.setClientId(10000000);
initRequest.setTransactionType(TransactionType.TOKEN);
initRequest.setTransactionAmount(new TransactionAmount(0));
initRequest.setClientRef("merchant_token_reference");
initRequest.setRedirect(new Redirect("https://merchant.com"));
PaymentInitResponse paymentInitResponse = client.payment().init(initRequest);
```

○ .NET

```
PaymentInitRequest initRequest = new PaymentInitRequest();
initRequest.clientId = 10000000;
initRequest.transactionType = Enums.TransactionType.TOKEN.ToString();
// $10.10 dollars
TransactionAmount transactionAmount = new TransactionAmount();
initRequest.transactionAmount = transactionAmount;
initRequest.clientRef = "merchant_token_reference";
Redirect redirect = new Redirect("https://merchant.com");
initRequest.redirect = redirect;
PaymentInitResponse paymentInitResponse = client.payment.init(initRequest);
```

○ PHP

```
initRequest = new PaymentInitRequest();
initRequest->setClientId(10000000);
initRequest->setTransactionType(TransactionType::$TOKEN);
// $10.10 dollars
$transactionAmount = new TransactionAmount(0);
initRequest->setTransactionAmount($transactionAmount);
initRequest->setClientRef("merchant_token_reference");
$redirect = new Redirect("https://merchant.com");
initRequest->setRedirect($redirect);
$initResponse = $client->getPayment()->init(initRequest);
```

7.1.2. Performing a purchase and obtaining token simultaneously

PURCHASE with tokenize Request

- JAVA

```
PaymentInitRequest initRequest = new PaymentInitRequest();
initRequest.setClientId(10000000);
initRequest.setTransactionType(TransactionType.PURCHASE);
// $10.10 dollars as the amount must be entered in cents
// "AUD" is the currency type . If this is not set then the default currency based
// on the country of the acquiring bank will apply
initRequest.setTransactionAmount(new TransactionAmount(1010,"AUD"));
initRequest.setClientRef("merchant_payment_reference");
initRequest.setRedirect(new Redirect("https://merchant.com"));
initRequest.setTokenize(true);
initRequest.setTokenReference("merchant_token_reference");
PaymentInitResponse paymentInitResponse = client.payment.init(initRequest);
```

- .NET

```
PaymentInitRequest initRequest = new PaymentInitRequest();
initRequest.clientId = 10000000;
initRequest.transactionType = Enums.TransactionType.PURCHASE.ToString();
// $10.10 dollars as the amount must be entered in cents
// "AUD" is the currency type . If this is not set then the default currency based
// on the country of the acquiring bank will apply
initRequest.transactionAmount = new TransactionAmount(1010,"AUD");
initRequest.clientRef = "merchant_reference";
initRequest.comment = "merchant_additional_data";
initRequest.redirect = new Redirect("https://merchant.com");
initRequest.tokenize = true;
initRequest.tokenReference = "merchant_token_reference";
PaymentInitResponse initResponse = client.payment.init(initRequest);
```

- PHP

```
initRequest = new PaymentInitRequest();
initRequest->setClientId(10000000);
initRequest->setTransactionType(TransactionType::$PURCHASE);
// $10.10 dollars as the amount must be entered in cents
// "AUD" is the currency type . If this is not set then the default currency based
// on the country of the acquiring bank will apply
$transactionAmount = new TransactionAmount(1010,"AUD");
initRequest->setTransactionAmount($transactionAmount);
initRequest->setClientRef("merchant_payment_reference");
$redirect = new Redirect("https://merchant.com");
initRequest->setRedirect($redirect);
initRequest->setTokenize(TRUE);
initRequest->setTokenReference("merchant_token_reference");
initRequest->getClient->getPayment()->init($paymentInitRequest);
```

Please note that paymentinit response , payment complete response message structure and procedure is the same as the previous description in section 4 referenced online technical guide page.

7.1.3. Tokenization Responses

Successful Responses

The following responses can be returned upon successful token generation.

Response Code	Response Text
00	SUCCESS
01	TOKEN ALREADY EXISTS IN DATABASE - CARD EXPIRY UPDATED.
02	PREAUTH TRANSACTION FAILED, HOWEVER TOKEN WAS STILL GENERATED.

Unsuccessful Responses

The following responses indicate a non-successful result.

Response Code	Response Text
20	INSUFFICIENT PARAMETERS SUPPLIED TO PERFORM FORM SUBMISSION
21	INTERNAL ERROR; PLEASE CONTACT SUPPORT.
22	INVALID CLIENTID USED WITH PROVIDED SSL CLIENT CERTIFICATE.
23	UNABLE TO FIND AND DELETE STORED CARD WITH SUPPLIED TOKEN.
24	UNABLE TO UPDATE STORED CARD - INVALID EXPIRY OR FAILED PREAUTH.
25	UNABLE TO FIND AND UPDATE STORED CARD WITH SUPPLIED TOKEN
26	UNABLE TO FIND STORED CARD WITH SUPPLIED TOKEN
27	PREAUTH TRANSACTION FAILED, NO TOKEN GENERATED
28	ACTION NOT PERMITTED FOR SUPPLIED CLIENTID.
29	TOO MANY RETRIEVAL ATTEMPTS - PLEASE CONTACT SUPPORT
30	INVALID CARD DATA SUPPLIED.

8. Making Payments Using Tokens

In order to make payments using tokens you need to use the Real Time Payments option provided by the same API.

Please note that you need to set up your credentials in the same manner as described before.

- JAVA

```
PaymentRealTimeRequest realTimeRequest = new PaymentRealTimeRequest();
CreditCard creditCard = new CreditCard();

// Enter TOKEN instead of actual card number
creditCard.setNumber("4564456687594564");
creditCard.setExpiry("1225"); *
realTimeRequest.setClientId(10000000);
realTimeRequest.setTransactionType(TransactionType.PURCHASE);
realTimeRequest.setCreditCard(creditCard);
// $10.10 dollars as the amount has to entered in cents
// "AUD" is the currency type . If this is not set then the default currency based
// on the country of the acquiring bank will apply
realTimeRequest.setTransactionAmount(new TransactionAmount(1010,"AUD"));
realTimeRequest.setClientRef("merchant_reference");
realTimeRequest.setComment("merchant_additional_data");
PaymentRealTimeResponse realTimeResponse = client.payment().realTime(realTimeRequest);
```

- NET

```
PaymentRealTimeRequest realTimeRequest = new PaymentRealTimeRequest();
CreditCard creditCard = new CreditCard();
// TOKEN
creditCard.number = "4564456687594564";
creditCard.expiry = "1225";
realTimeRequest.clientId = 10000000;
realTimeRequest.transactionType = Enums.TransactionType.PURCHASE.ToString();
realTimeRequest.creditCard= creditCard;
// $10.10 dollars
// "AUD" is the currency type . If this is not set then the default currency based
// on the country of the acquiring bank will apply
realTimeRequest.transactionAmount = new TransactionAmount(1010,"AUD");
realTimeRequest.clientRef = "merchant_reference";
realTimeRequest.comment = "merchant_additional_data";
PaymentRealTimeResponse realTimeResponse = client.payment.realTime(realTimeRequest);
```

- PHP

```
$realTimeRequest = new PaymentRealTimeRequest();
$creditCard = new CreditCard();
// TOKEN
$creditCard->setNumber("4564456687594564");
$creditCard->setExpiry("1225");
$realTimeRequest->setClientId(10000000);
$realTimeRequest->setTransactionType(TransactionType::$PURCHASE);
$realTimeRequest->setCreditCard($creditCard);
// $10.10 dollars
// "AUD" is the currency type . If this is not set then the default currency based
// on the country of the acquiring bank will apply
$transactionAmount = new TransactionAmount(1010,"AUD");
$realTimeRequest->setTransactionAmount($transactionAmount);
$realTimeRequest->setClientRef("merchant_reference");
$realTimeRequest->setComment("merchant_additional_data");
$realTimeResponse = $client->getPayment()->realTime($realTimeRequest);
```

Support and Troubleshooting

If you need assistance or encounter any issues during integration, please contact our support team at support@bancstac.com or call us on +94 115 143 143